

Homework 1

CS 4033/5033, Machine Learning Fundamentals, Fall 2026

Ridwan Hoque

Part 1: AN Representation

Preliminary: why we augment the input vector

In a standard neural network layer, the math that we need to perform in order to calculate the output y requires two separate operations: a matrix multiplication and a vector addition.

$$y = W\mathbf{x} + b,$$

where $\mathbf{x}$ is the input vector (for example, the features or the data points), W is the weight matrix, and b is the bias vector, which shifts the activation function.

In code and in hardware such as GPUs, having to multiply and then add separately slows down processing, because we have to perform two distinct memory operations rather than one.

The solution: the augmented vector trick. By augmenting the input vector with a constant 1, we are able to fuse the weight and the bias together into a single, highly optimized matrix multiplication step.

Step 1: augment the input vector. We need to take the original input vector $\mathbf{x}$ and append a 1 at the very bottom:

$$\mathbf{x}_{\text{aug}} = \begin{bmatrix} x_1 \\ x_2 \\ \mathbf{1} \end{bmatrix}.$$

Step 2: augment the weight matrix. We need to take the original weight matrix W and append the bias vector b as an extra column at the end:

$$W_{\text{aug}} = \begin{bmatrix} w_{11} & w_{12} & \mathbf{b}_1 \\ w_{21} & w_{22} & \mathbf{b}_2 \end{bmatrix}.$$

The clean result. The neural network now only has to perform one single multiplication:

$$y = W_{\text{aug}} \cdot \mathbf{x}_{\text{aug}}.$$

If we write this out, the math naturally multiplies the weights by the inputs and automatically adds the bias at the end, because the bias column gets multiplied by the constant 1.

This is exactly the convention that the rest of this assignment uses. Throughout, we have to work with the augmented input $\mathbf{p} = (1, x_1, x_2)$ and the augmented weight vector $\mathbf{w} = (w_0, w_1, w_2)$, so that the threshold test reduces to the single condition $\mathbf{w} \cdot \mathbf{p} \geq 0$.

1.1 Draw the AN

The artificial neuron has an augmented input vector $\mathbf{p} = (x_0, x_1, x_2) = (1, x_1, x_2)$ and a weight vector $\mathbf{w} = (w_0, w_1, w_2) = (0.5, 1.0, -0.3)$. The summation unit computes

$$\text{net} = \mathbf{w} \cdot \mathbf{p} = w_0x_0 + w_1x_1 + w_2x_2 = 0.5 + 1.0x_1 - 0.3x_2,$$

and the step activation produces the output

$$a = f(\text{net}) = \begin{cases} 1, & \text{net} \geq 0, \\ 0, & \text{net} < 0. \end{cases}$$

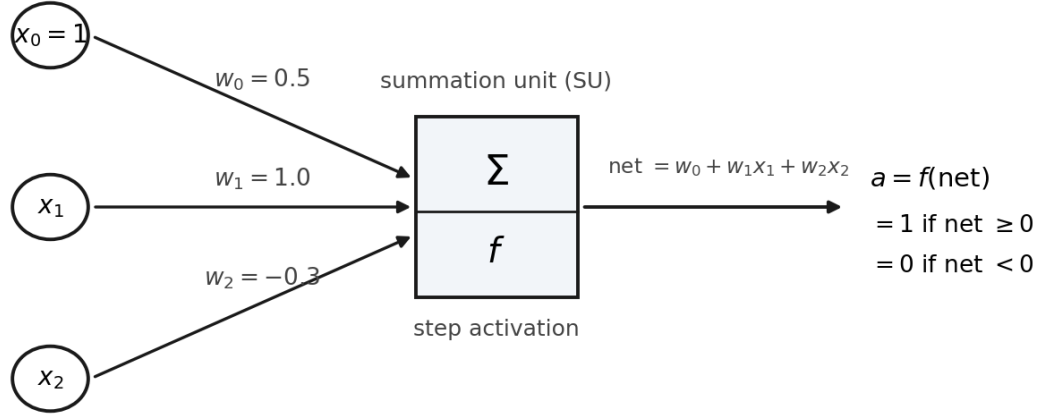


Figure 1: The artificial neuron. The constant input $x_0 = 1$ carries the bias weight w_0 , so the threshold is absorbed into the weight vector and the test becomes simply $\mathbf{w} \cdot \mathbf{p} \geq 0$.

1.2 Decision boundary

The boundary is the locus where the neuron switches its output; that is, where $\text{net} = 0$:

$$0.5 + x_1 - 0.3x_2 = 0 \quad \Longleftrightarrow \quad x_2 = \frac{0.5 + x_1}{0.3} = \frac{5 + 10x_1}{3}.$$

This is a straight line with slope $10/3 \approx 3.33$, with x_2 -intercept $5/3 \approx 1.67$ (at $x_1 = 0$) and x_1 -intercept -0.5 (at $x_2 = 0$).

Which side is the “front”? The vector of non-bias weights $(w_1, w_2) = (1.0, -0.3)$ is normal to the line, and it points into the half-plane where net increases; that is, toward Class 1. Moving in the direction $(1.0, -0.3)$ means moving right and slightly down, so the Class 1 (“front”) side is the lower-right half-plane. As a check, we need to evaluate the origin $(0, 0)$, which gives $\text{net} = 0.5 \geq 0$, so the origin lies on the Class 1 side.

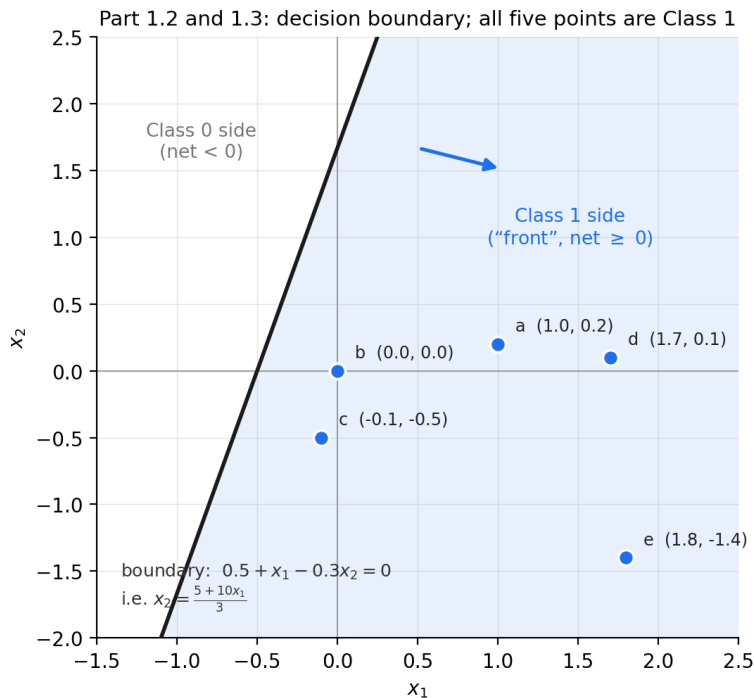


Figure 2: The decision boundary $0.5 + x_1 - 0.3x_2 = 0$. The shaded lower-right half-plane is the “front” (Class 1, $\text{net} \geq 0$); the arrow is the weight normal $(w_1, w_2) = (1.0, -0.3)$. The five points of question 1.3 are plotted, and all five of them fall on the Class 1 side.

1.3 Add and label the five points

	Point (x_1, x_2)	$\text{net} = 0.5 + x_1 - 0.3x_2$	Class $a = f(\text{net})$
(a)	(1.0, 0.2)	$0.5 + 1.0 - 0.06 = 1.44$	1
(b)	(0.0, 0.0)	$0.5 + 0.0 - 0.00 = 0.50$	1
(c)	(-0.1, -0.5)	$0.5 - 0.1 + 0.15 = 0.55$	1
(d)	(1.7, 0.1)	$0.5 + 1.7 - 0.03 = 2.17$	1
(e)	(1.8, -1.4)	$0.5 + 1.8 + 0.42 = 2.72$	1

Table 1: Every net value is non-negative, so this AN assigns *all five* points to Class 1. The points are plotted in Figure 2.

1.4 A new point that forces a misclassification

The new data item. We have to add the labeled point

$$(0.9, -0.7) \text{ with label Class 0.}$$

Why no weights can work. All five of the original points are Class 1, and the new point is the *midpoint* of two of them:

$$(0.9, -0.7) = \frac{1}{2}[(0.0, 0.0) + (1.8, -1.4)] = \frac{1}{2}(b + e).$$

Now we need to consider *any* weight vector $\mathbf{w} = (w_0, w_1, w_2)$ whatsoever. Because $\text{net}(\mathbf{p}) = \mathbf{w} \cdot \mathbf{p}$ is an affine function of the point, and because the augmented coordinate $x_0 = 1$ is the same for all three points, the value of net at the midpoint is the average of the two end values:

$$\text{net}\left(\frac{b+e}{2}\right) = \frac{1}{2}[\text{net}(b) + \text{net}(e)].$$

Suppose such a $\mathbf{w}$ classified points b and e correctly. Both of them are Class 1, so $\text{net}(b) \geq 0$ and $\text{net}(e) \geq 0$. Then

$$\text{net}\left(\frac{b+e}{2}\right) = \frac{1}{2}[\underbrace{\text{net}(b)}_{\geq 0} + \underbrace{\text{net}(e)}_{\geq 0}] \geq 0,$$

so the AN outputs $a = 1$ at $(0.9, -0.7)$, but that point is labeled Class 0. This is a contradiction. Hence, for every possible $\mathbf{w}$, at least one of $\{b, e, (0.9, -0.7)\}$ has to be misclassified.

The general principle. A single AN can only realize a *linear* separator, and a linear separator exists if and only if the convex hulls of the two classes do not intersect. Any linear half-plane that encloses all of the surrounding Class 1 vertices must also enclose all of the interior points. Assigning an interior point to Class 0 therefore creates a data set that is not linearly separable, in the same way that the XOR problem is not linearly separable, and this makes it impossible for any set of weights (w_0, w_1, w_2) to classify all of the points correctly. The new point lies strictly inside the convex hull of the Class 1 points, in fact on the chord joining b and e , so the two hulls overlap.

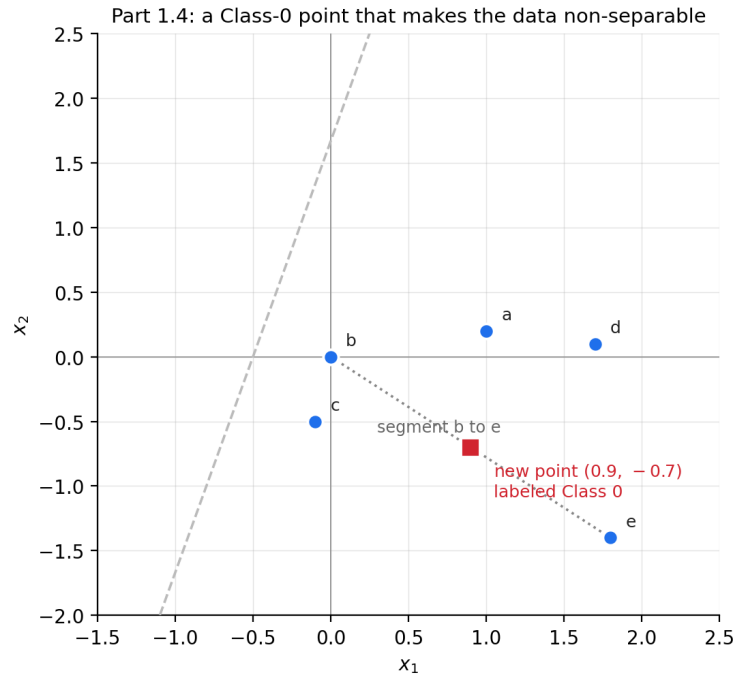


Figure 3: The Class-0 point $(0.9, -0.7)$ sits on the segment joining the Class-1 points $b = (0, 0)$ and $e = (1.8, -1.4)$. Any line putting b and e on its positive side must also put their midpoint there, so no single AN is able to classify all six points correctly.

1.5 Changing f to output $\{1, -1\}$

What changes. *Nothing about the decision boundary.* It remains the same line $0.5 + x_1 - 0.3x_2 = 0$, with the same slope, the same intercepts, and the same “front” side. The only change is the

numeric label attached to the negative half-plane: points that used to produce output 0 now produce output -1 .

Why. The decision boundary is determined entirely by *where the activation function switches*, and not by the values that it switches between. Both step functions switch at exactly the same argument, $\text{net} = 0$:

$$f_{\text{old}}(\text{net}) = \begin{cases} 1, & \text{net} \geq 0 \\ 0, & \text{net} < 0 \end{cases} \quad f_{\text{new}}(\text{net}) = \begin{cases} 1, & \text{net} \geq 0 \\ -1, & \text{net} < 0 \end{cases}$$

Since the geometry of the boundary comes from solving $\text{net} = 0$, an equation that involves only $\mathbf{w}$ and $\mathbf{p}$ and not f , and since f is a monotone function of net in both cases with its discontinuity at the same place, the partition of the input plane into two half-planes is identical. Only the *encoding* of the negative class changes, from 0 to -1 .

This does matter elsewhere. A $\{-1, 1\}$ encoding makes the quantity $t - a$ in a learning rule equal to ± 2 instead of ± 1 , which rescales the size of a weight update. However, it does not move the boundary for a *fixed* weight vector.

1.6 Changing f to the logistic sigmoid

1. The raw score (net). Before we apply any activation function, the neuron calculates a single number called net , which is a weighted sum of the inputs:

$$\text{net} = 0.5 + 1.0x_1 - 0.3x_2.$$

We need to think of net as a score:

- Positive score ($\text{net} > 0$): points on the Class 1 side.
- Negative score ($\text{net} < 0$): points on the Class 0 side.
- Zero score ($\text{net} = 0$): points sitting directly on the dividing line.

2. The step function (hard wall). With the step function, the neuron makes a harsh, all-or-nothing decision:

- If $\text{net} \geq 0 \rightarrow$ output 1 (Class 1).
- If $\text{net} < 0 \rightarrow$ output 0 (Class 0).

The switch point, where the output jumps from 0 to 1, happens exactly at $\text{net} = 0$.

3. The sigmoid function (smooth ramp). The sigmoid function replaces that sharp jump with a smooth curve that outputs numbers between 0 and 1:

$$\text{output} = \sigma(\text{net}) = \frac{1}{1 + e^{-\text{net}}}.$$

To convert the continuous sigmoid outputs into binary classes, standard classification uses a 0.5 threshold:

- If $\text{output} \geq 0.5 \rightarrow$ predict Class 1.
- If $\text{output} < 0.5 \rightarrow$ predict Class 0.

4. Why the line stays in the exact same spot. We need to look at what happens to the sigmoid equation when the input score is zero:

$$\sigma(0) = \frac{1}{1 + e^0} = \frac{1}{1 + 1} = \mathbf{0.5}.$$

- When $\text{net} = 0$, the sigmoid output is exactly 0.5.
- When $\text{net} > 0$, the sigmoid output is greater than 0.5 (Class 1).
- When $\text{net} < 0$, the sigmoid output is less than 0.5 (Class 0).

Because the classification threshold is 0.5, we still have to find the dividing line between Class 1 and Class 0 by solving $\text{net} = 0$. Since the equation $0.5 + x_1 - 0.3x_2 = 0$ has not changed, the geometric line plotted on the graph remains identical.

Summary of what changes and what stays the same.

- **Stays the same:** the geometric boundary line on the graph ($\text{net} = 0$).
- **Changes:** instead of a sharp cliff at the boundary, the sigmoid creates a smooth confidence gradient. Points far from the line get outputs close to 1 or 0, which means high confidence, while points right next to the line get outputs close to 0.5, which means low confidence and an undecided result.

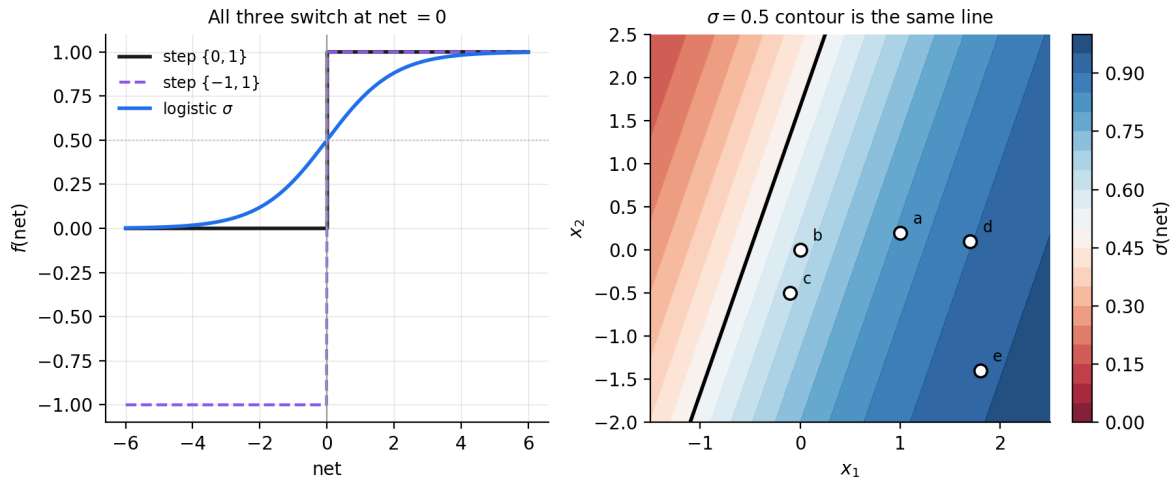


Figure 4: Left: all three activation functions switch at $\text{net} = 0$, so all three give the same boundary. The step functions differ only in the value assigned to the negative side, and the sigmoid replaces the jump with a smooth ramp. Right: $\sigma(\text{net})$ over the input plane, where the black line is the $\sigma = 0.5$ contour, identical to the boundary of Figure 2.

1.7 Classification of the five points under the sigmoid

	Point	net	$\sigma(\text{net}) = \frac{1}{1+e^{-\text{net}}}$	Class (threshold 0.5)
(a)	(1.0, 0.2)	1.44	0.808	1
(b)	(0.0, 0.0)	0.50	0.622	1
(c)	(-0.1, -0.5)	0.55	0.634	1
(d)	(1.7, 0.1)	2.17	0.897	1
(e)	(1.8, -1.4)	2.72	0.938	1

Answer. The classifications *do not change*, and all five points are still Class 1. What changes is that the AN no longer treats them as equivalent. Under the step function, every one of these points produced the identical output 1. Under the sigmoid, the output grades them by how far they sit from the boundary. Point *e*, which is deepest inside the Class-1 region, gets 0.938, while points *b* and *c* sit close to the line and get only about 0.62 to 0.63, which means the neuron is barely committed to them. If we had to use a stricter threshold than 0.5, for example 0.7, then *b* and *c* would flip to Class 0, which the step function could never express.

Part 2: AN Learning

Throughout this part, we have to apply the learning rule

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + (t - a) \mathbf{p},$$

where $\mathbf{p} = (1, x_1, x_2)$ is the augmented input, $t \in \{0, 1\}$ is the target, and $a = f(\text{net})$ is the actual output of the step function of question 1.1. The starting weights are $\mathbf{w} = (0.5, 1.0, -0.3)$.

We should note that because $t, a \in \{0, 1\}$, the factor $(t - a)$ can only take three values:

$$t - a = \begin{cases} 0 & \text{correct, so we do not need to update,} \\ +1 & \text{predicted 0, should be 1, so we need to add } \mathbf{p}, \\ -1 & \text{predicted 1, should be 0, so we need to subtract } \mathbf{p}. \end{cases}$$

2.1 Weight updates, one pass through the data

(a) **Class 0:** $(-0.1, -1.0)$, so $\mathbf{p} = (1, -0.1, -1.0)$ and $t = 0$.

$$\begin{aligned} \text{net} &= 0.5 + (1.0)(-0.1) + (-0.3)(-1.0) = 0.5 - 0.1 + 0.3 = 0.70 \geq 0 \Rightarrow a = 1. \\ t - a &= 0 - 1 = -1 \quad (\text{misclassified, so we need to update}) \\ \mathbf{w}_{\text{new}} &= (0.5, 1.0, -0.3) + (-1)(1, -0.1, -1.0) = (-0.5, 1.1, 0.7). \end{aligned}$$

(b) **Class 0:** $(0.2, -0.9)$, so $\mathbf{p} = (1, 0.2, -0.9)$ and $t = 0$.

$$\begin{aligned} \text{net} &= -0.5 + (1.1)(0.2) + (0.7)(-0.9) = -0.5 + 0.22 - 0.63 = -0.91 < 0 \Rightarrow a = 0. \\ t - a &= 0 - 0 = 0 \quad (\text{correct, so no change}) \\ \mathbf{w}_{\text{new}} &= (-0.5, 1.1, 0.7). \end{aligned}$$

(c) **Class 1:** (0.6, 0.8), so $\mathbf{p} = (1, 0.6, 0.8)$ and $t = 1$.

$$\begin{aligned}\text{net} &= -0.5 + (1.1)(0.6) + (0.7)(0.8) = -0.5 + 0.66 + 0.56 = 0.72 \geq 0 \Rightarrow a = 1. \\ t - a &= 1 - 1 = 0 \quad (\text{correct, so no change}) \\ \mathbf{w}_{\text{new}} &= (-0.5, 1.1, 0.7).\end{aligned}$$

(d) **Class 1:** (0.0, 0.0), so $\mathbf{p} = (1, 0.0, 0.0)$ and $t = 1$.

$$\begin{aligned}\text{net} &= -0.5 + (1.1)(0) + (0.7)(0) = -0.50 < 0 \Rightarrow a = 0. \\ t - a &= 1 - 0 = +1 \quad (\text{misclassified, so we need to update}) \\ \mathbf{w}_{\text{new}} &= (-0.5, 1.1, 0.7) + (+1)(1, 0, 0) = (0.5, 1.1, 0.7).\end{aligned}$$

Summary of pass 1. The following table collects the whole pass.

Step	$\mathbf{p}$	t	net	a	$\mathbf{w}$ after the step	
	(start)				(0.5, 1.0, -0.3)	
(a)	(1, -0.1, -1.0)	0	+0.70	1	(-0.5, 1.1, 0.7)	<i>updated</i>
(b)	(1, 0.2, -0.9)	0	-0.91	0	(-0.5, 1.1, 0.7)	no change
(c)	(1, 0.6, 0.8)	1	+0.72	1	(-0.5, 1.1, 0.7)	no change
(d)	(1, 0.0, 0.0)	1	-0.50	0	(0.5, 1.1, 0.7)	<i>updated</i>

We should also note that the update of $+\mathbf{p}$ for the point (0,0) changes *only* the bias weight w_0 , since both real coordinates of that point are zero. Geometrically, the boundary gets translated without being rotated.

2.2 Data and decision boundary after each update

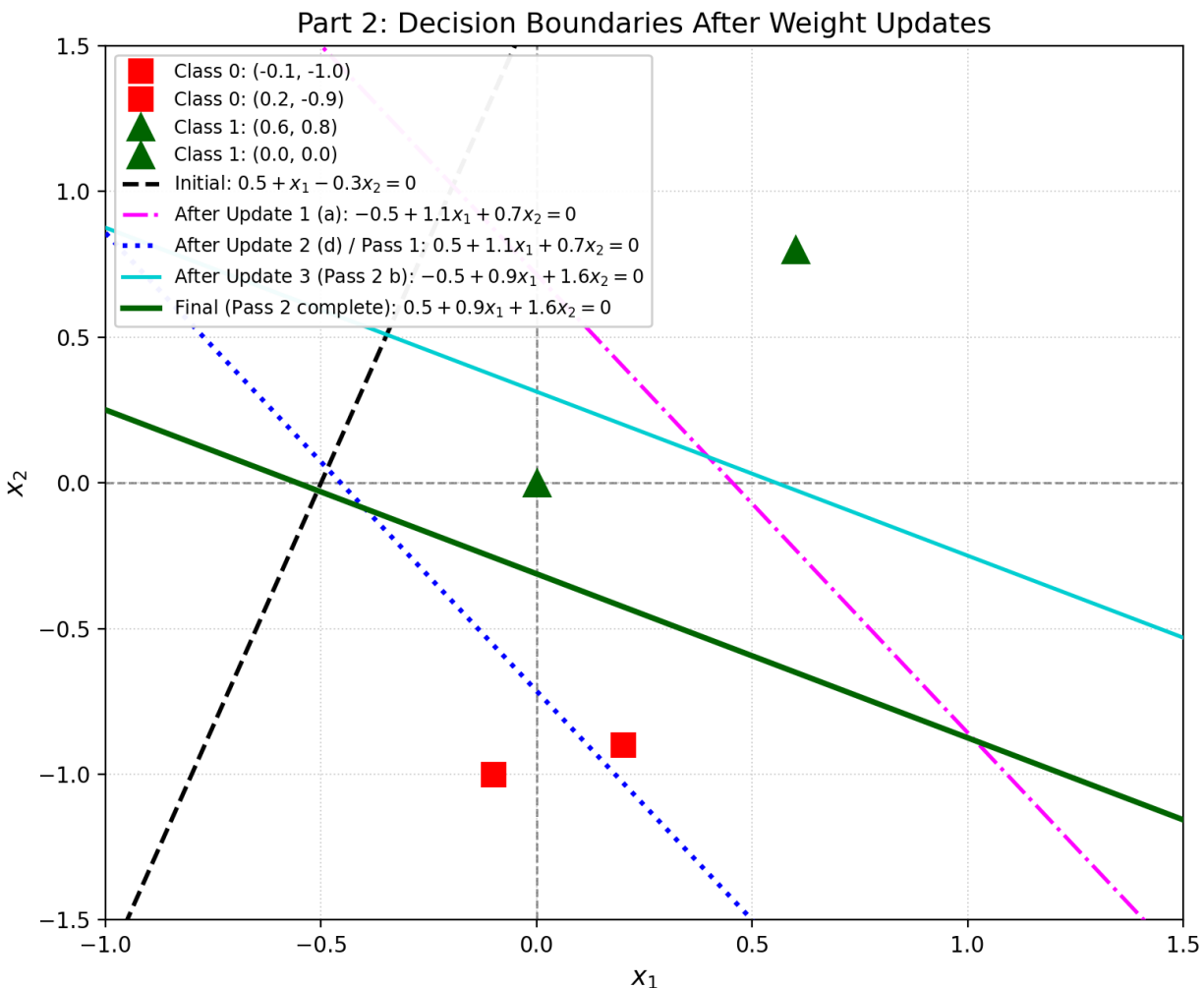


Figure 5: The four training points together with the decision boundary after every weight update. The black dashed line is the initial boundary, and each subsequent line shows the boundary immediately after one update. The solid dark green line is the final boundary, which classifies all four points correctly.

2.3 Which points are correct after one full pass?

If we evaluate all four points with the post-pass weights $\mathbf{w} = (0.5, 1.1, 0.7)$:

	Point	t	$\text{net} = 0.5 + 1.1x_1 + 0.7x_2$	a	Correct?
(a)	$(-0.1, -1.0)$	0	$0.5 - 0.11 - 0.70 = -0.31$	0	yes
(b)	$(0.2, -0.9)$	0	$0.5 + 0.22 - 0.63 = +0.09$	1	no
(c)	$(0.6, 0.8)$	1	$0.5 + 0.66 + 0.56 = +1.72$	1	yes
(d)	$(0.0, 0.0)$	1	$0.5 + 0.00 + 0.00 = +0.50$	1	yes

Answer. Points (a), (c) and (d) are correctly classified, while point (b) is **misclassified**, at $\text{net} = +0.09$, which is just barely on the wrong side of the line.

What this tells us about the learning rule. Online, or incremental, learning updates the weights instance by instance. Adjusting the weights in order to correct one sample, such as point (d), can shift the decision boundary in such a way that a previously correct sample, such as point (b), becomes misclassified. Thus, a single pass is rarely sufficient, and we need multiple passes, or epochs, in order to reach full convergence.

2.4 How many more passes are needed?

Answer: one more pass. After a second pass, all four points are correctly classified, and a third pass produces no updates at all, which confirms convergence.

Work for pass 2, starting from $\mathbf{w} = (0.5, 1.1, 0.7)$.

- (a) $\mathbf{p} = (1, -0.1, -1.0)$, $t = 0$: $\text{net} = 0.5 - 0.11 - 0.70 = -0.31 < 0 \Rightarrow a = 0$, $t - a = 0$ (no change)
- (b) $\mathbf{p} = (1, 0.2, -0.9)$, $t = 0$: $\text{net} = 0.5 + 0.22 - 0.63 = +0.09 \geq 0 \Rightarrow a = 1$, $t - a = -1$
 $\mathbf{w} \leftarrow (0.5, 1.1, 0.7) - (1, 0.2, -0.9) = (-0.5, 0.9, 1.6)$
- (c) $\mathbf{p} = (1, 0.6, 0.8)$, $t = 1$: $\text{net} = -0.5 + 0.54 + 1.28 = +1.32 \geq 0 \Rightarrow a = 1$, $t - a = 0$ (no change)
- (d) $\mathbf{p} = (1, 0, 0)$, $t = 1$: $\text{net} = -0.5 < 0 \Rightarrow a = 0$, $t - a = +1$
 $\mathbf{w} \leftarrow (-0.5, 0.9, 1.6) + (1, 0, 0) = (\mathbf{0.5}, \mathbf{0.9}, \mathbf{1.6})$

	Point	t	$\text{net} = 0.5 + 0.9x_1 + 1.6x_2$	a	Correct?
(a)	$(-0.1, -1.0)$	0	$0.5 - 0.09 - 1.60 = -1.19$	0	yes
(b)	$(0.2, -0.9)$	0	$0.5 + 0.18 - 1.44 = -0.76$	0	yes
(c)	$(0.6, 0.8)$	1	$0.5 + 0.54 + 1.28 = +2.32$	1	yes
(d)	$(0.0, 0.0)$	1	$0.5 + 0.00 + 0.00 = +0.50$	1	yes

Check of all four points with $\mathbf{w} = (0.5, 0.9, 1.6)$: All four of them are correct, so pass 3 encounters $t - a = 0$ at every point and leaves $\mathbf{w} = (0.5, 0.9, 1.6)$ unchanged. The algorithm has converged. On the scale suggested in the question, the answer is **one**.

2.5 Effect of introducing a learning rate η

If we introduce a learning rate $\eta \in (0, 1]$, the update rule becomes

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta(t - a)\mathbf{p}.$$

1. **Controlled step size.** Scaling the updates by η moderates the magnitude of the weight adjustment that we need to make at each step.
2. **Improved stability.** Smaller updates prevent wild oscillations of the boundary, which keeps previously correctly classified points stable.
3. **Convergence rate.** While small learning rates ensure smoother convergence, they may increase the total number of passes that we have to perform in order to achieve full separation.

Appendix: Handwritten Work

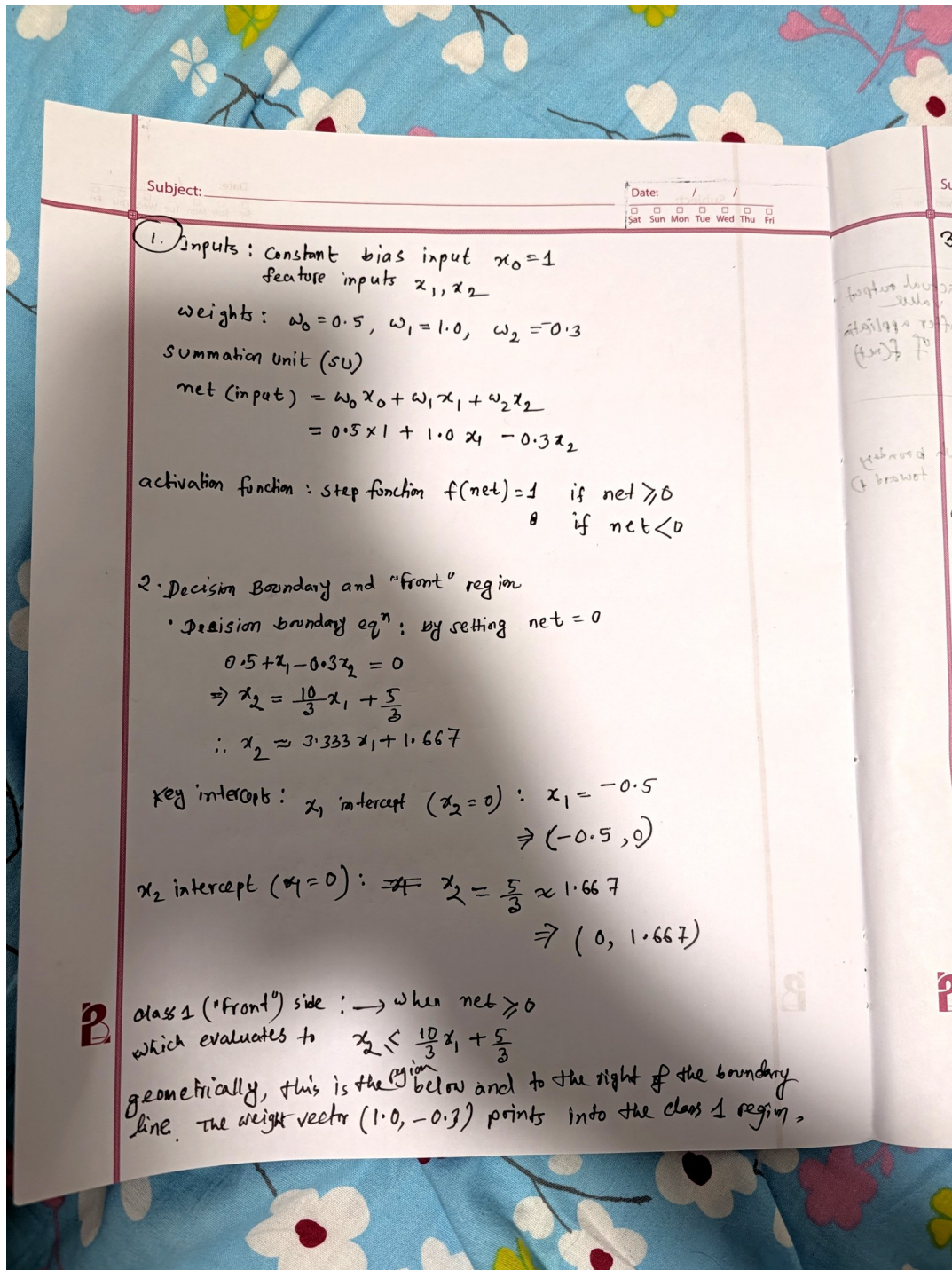


Figure 6: Handwritten work, page 1: questions 1.1 and 1.2.

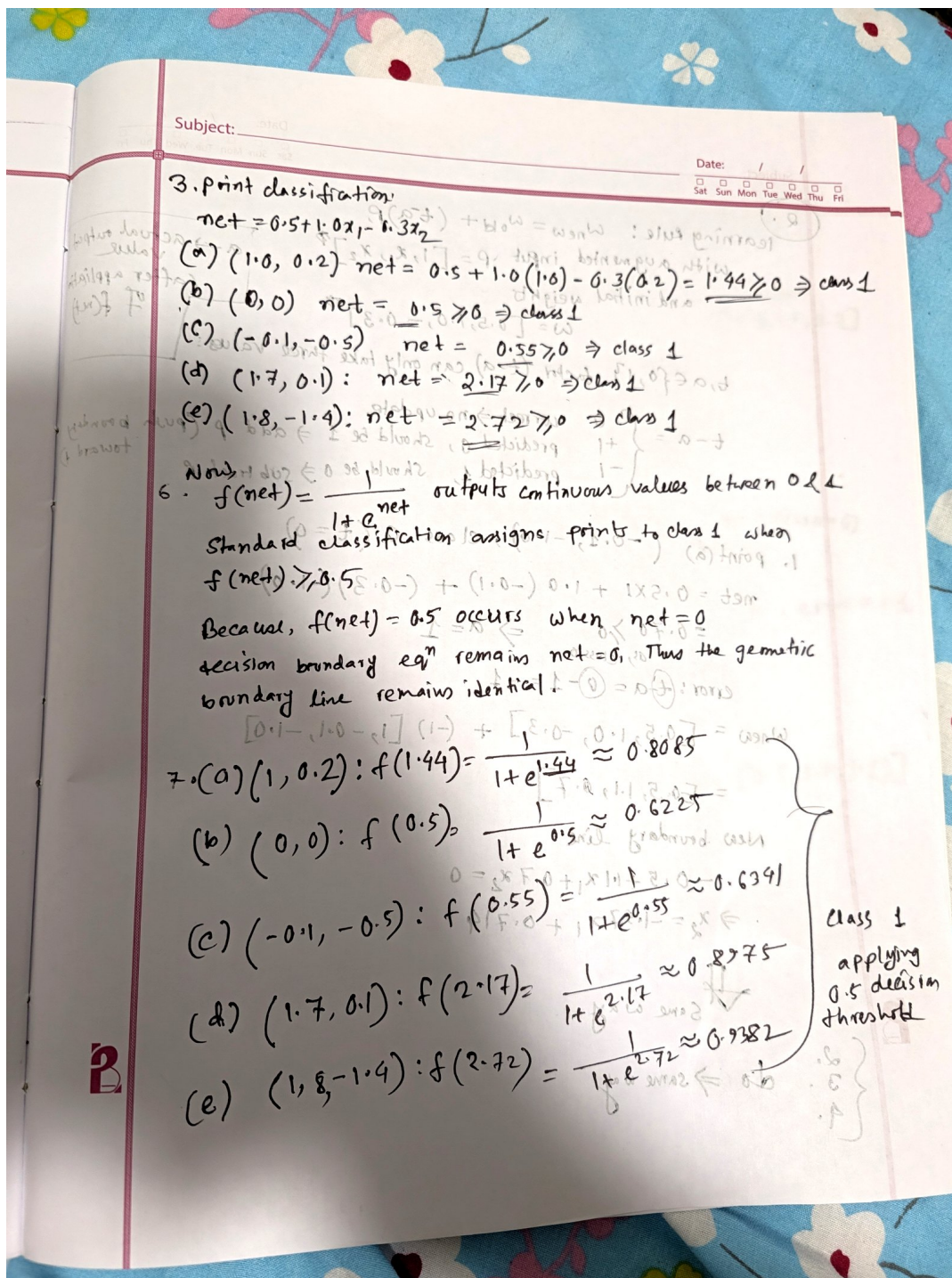


Figure 7: Handwritten work, page 2: questions 1.3, 1.6 and 1.7.

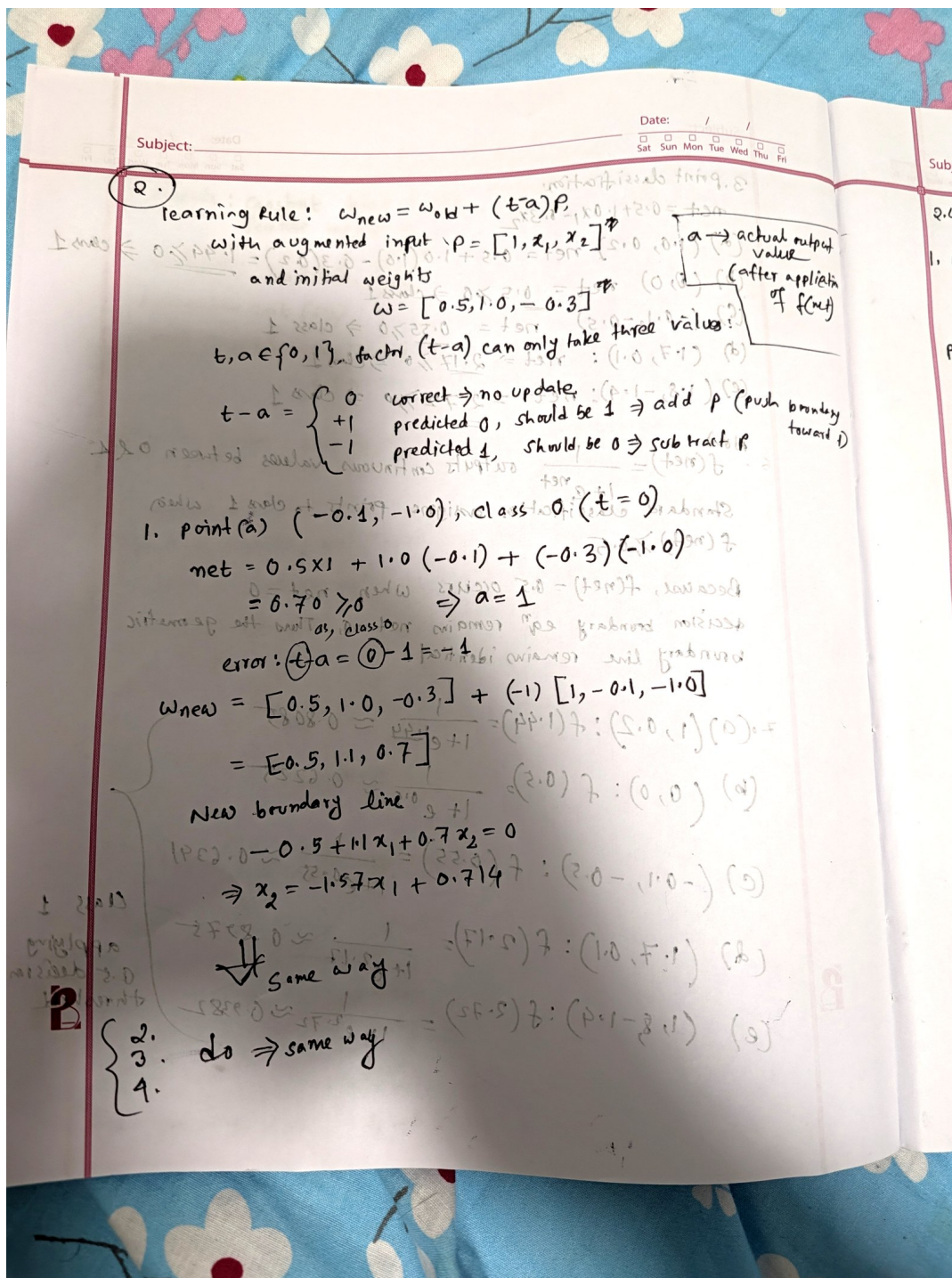


Figure 8: Handwritten work, page 3: question 2.1, the learning rule and the update at point (a).

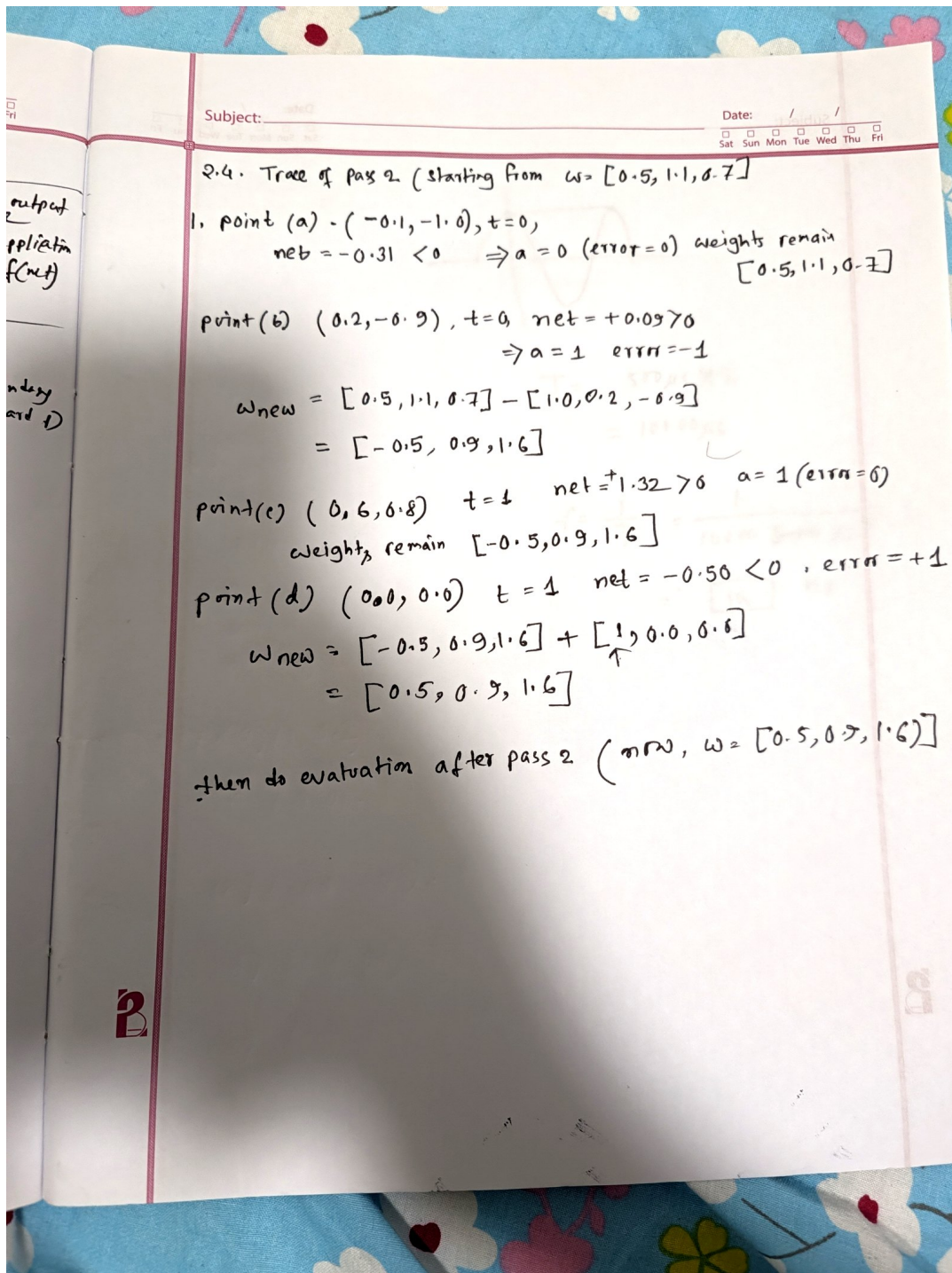


Figure 9: Handwritten work, page 4: question 2.4, the trace of pass 2.